

Photon-Noise Sonifier: 3-Hour Build Guide

TSL2591 + Raspberry Pi → light noise → random numbers → music

Youssef Nassar · everything runs at 3.3 V

The idea in three lines

- Phone light goes through paper and a small hole in the box into the TSL2591. You record **one run**; the phone's distance is the only thing you adjust.
- Every reading jitters by a few counts (photon shot noise plus sensor noise). The **last digit** of each reading is a coin flip: that is your random-number generator.
- The random bits become numbers, dice rolls and music. Not a certified quantum RNG, and the report says so.

Time plan

Clock	Step	Output
0:00–0:20	1 Software + I ² C	scripts run in <code>--simulate</code>
0:20–0:30	2 Wire + check	live numbers from the sensor
0:30–1:00	3 Hole + enclosure + leak test	dark reading that ignores room light
1:00–1:15	4 The run (5 min, set up Sonic Pi meanwhile)	<code>data/run.csv</code>
1:15–1:20	5 Analyze	random bits, dice, tests, report files
1:20–1:50	6 Sonify + record demo	live music, video
1:50–2:30	7 Report on Overleaf	finished PDF
2:30–3:00	buffer / extra sound waves	

Parts

You have	Used for
Adafruit TSL2591	the detector (I ² C address 0x29)
STEMMA QT cable, jumpers, breadboard	sensor ↔ Pi header
Opaque enclosure, black foam tape	light-tight box
Raspberry Pi	acquisition, analysis, music
330 Ω resistor	not needed (it is an LED current limiter; see note)
From the house	a pen (to poke the hole), printer paper, black electrical tape, craft knife or drill, phone with flashlight (your light source), a stack of books

No LED needed: a phone flashlight or window daylight is the source. The one requirement is that it doesn't move during a capture. *Optional*, if you ever grab a 5 mm LED: GPIO18 → 330 Ω → LED → GND gives $I \approx (3.3 - 2.0)/330 \approx 4 \text{ mA}$, a steady source the Pi controls.

1 Software and I²C

0:00–0:20

Copy the `photon-sonifier` folder to the Pi (USB stick, `scp`, or download), then:

```
cd ~/photon-sonifier
sudo apt update
sudo apt install -y i2c-tools python3-smbus2 python3-numpy python3-matplotlib sonic-pi
sudo raspi-config nonint do_i2c 0 # enable I2C
```

```
sudo reboot
```

If `python3-smbus2` is not found, install `python3-smbus` instead (the driver accepts either). Rehearse the whole pipeline without hardware, then delete the fake data:

```
cd ~/photon-sonifier
python3 collect.py run --simulate && python3 analyze.py
rm -r data # simulated data must never reach the report
```

✓ **Done when:** `analyze.py` prints test results and dice rolls.

2 Wiring and first reading

0:20–0:30

Pi **powered off**. STEMMA QT wire colours are standard:

Wire	Signal	Pi header pin
Red	3.3 V	pin 1
Black	GND	pin 6
Blue	SDA	pin 3 (GPIO2)
Yellow	SCL	pin 5 (GPIO3)

Pi GPIO header, pins 1–6

1 3V3	2 5V
3 SDA	4 5V
5 SCL	6 GND

Which end does your cable have on the Pi side?

- **Female sockets:** push straight onto pins 1, 3, 5, 6.
- **Male pins:** into the breadboard, four jumpers to the Pi.
- **Another JST plug:** solder the breakout's header strip and use four female–female jumpers (VIN, GND, SDA, SCL on the breakout's pin row), or use a Pi STEMMA QT adapter.

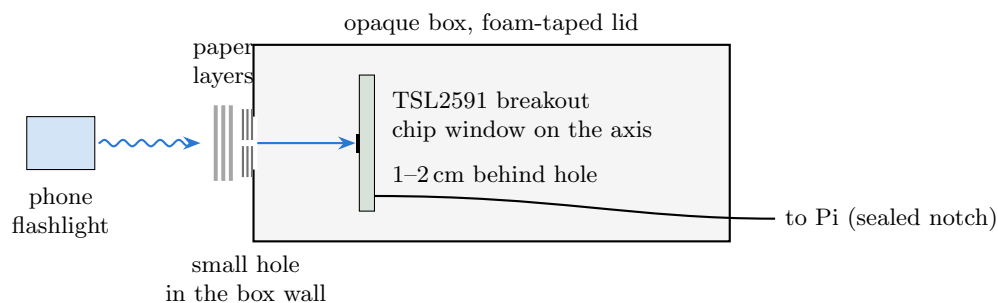
Use pin 1 (3.3 V), not 5 V: it keeps the I²C pull-ups at the Pi's logic level.

```
i2cdetect -y 1 # expect "29" in the grid
python3 collect.py check # live ch0/ch1; Ctrl+C to stop
```

✓ **Done when:** 29 appears, and ch0 drops when you cover the sensor with a finger. Rate ≈ 8 Hz.

3 Hole and enclosure

0:30–1:00



3a. Hole. Poke one small hole (pen tip, 2–4 mm) in a wall of the box. Cut a small notch for the cable at the lid edge and line the lid seam with foam tape.

3b. Mount. Place the breakout 1–2 cm behind the hole with the small clear chip facing it, and fix it with foam tape.

3c. Paper. Tape a few layers of printer paper over the *outside* of the hole. More layers = dimmer. Change the number of layers if the phone can't get the reading into range.

3d. Light source. Phone flashlight on a stack of books, pointing at the hole. Its **distance is your**

only brightness knob. Nothing moves during the run.

3e. Leak test. Tape the hole shut with 3 layers of black tape, close the lid, run `python3 collect.py check`, then shine the flashlight at every seam and the cable notch.

✓ **Done when:** ch0 sits at a few counts and does not move when you light the seams. If it jumps, add tape or foam where it jumped.

4 The run

1:00–1:15

Remove the black tape from the hole, then:

```
python3 collect.py run # 5 minutes; --minutes 10 for more random bits
```

It shows ch0 live with a hint (*move closer / move farther / good*). Slide the phone until ch0 is between **3 000 and 25 000**, rest it on the books, press **Enter**, and don't touch anything until it finishes. If it never gets bright enough even up close, remove one paper layer.

While it records: open Sonic Pi, paste `sonic_pi.rb` into a buffer, press **Run**; open Overleaf.

✓ **Done when:** `data/run.csv` saved.

5 Analyze

1:15–1:20

```
python3 analyze.py
```

How the random numbers are made: the last bit of each reading (both channels) is a raw coin flip. A *von Neumann extractor* fixes any bias: read bits in pairs, $01 \rightarrow 0$, $10 \rightarrow 1$, throw away 00 and 11. About a quarter of the bits survive. Three clean bits make a number 0–7; keeping only 0–5 gives fair dice.

It prints four randomness tests (pass if $p \geq 0.01$), the first 64 bits and 20 dice rolls, and saves `data/random.bits.txt` and `data/random.bytes.bin`. It also prints the **ch0/ch1 noise correlation**: near 0 means the noise is independent on each photodiode (shot + sensor noise); large means the phone light is flickering.

✓ **Done when:** all four tests pass. If the noise is under 1.5 counts or tests fail, move the phone closer and redo Step 4.

6 Sonify and record the demo

1:20–1:50

With the Sonic Pi buffer running:

```
python3 sonify.py
```

Each 4 debiased noise bits become one bell note (3 bits pick one of 8 notes of A minor pentatonic, 1 bit picks short or long). Cover the hole: the pad darkens, notes keep coming from the dark noise. Record 60s on your phone showing the box, the terminal and the sound.

- **No sound?** Pi 5 has no headphone jack: use HDMI audio or a USB/Bluetooth speaker. In Sonic Pi, the Cues panel should show `/osc:127.0.0.1:.../photon/note`.
- **Sonic Pi on your laptop instead:** tick “Allow incoming OSC from other computers” in its I/O settings, then `python3 sonify.py --host <laptop-ip>`.
- **Guaranteed fallback:** `python3 sonify.py --wav demo.wav` renders 60s of music from your recorded run, no Sonic Pi needed.

✓ **Done when:** you have a demo video (or `demo.wav`) and a link to it.

7 Report

1:50–2:30

`analyze.py` already wrote `report_overleaf.zip`. Overleaf → New Project → Upload Project → `report_overleaf.zip` → Recompile. Edit the two lines at the top of `report.tex`: `\authorNames` and `\demoLink`. Every number, table, figure and verdict sentence comes from your data; the code is in the appendix.

✓ **Done when:** no red “[run analyze.py]” placeholders and no red SIMULATED banner. Download the PDF.

Worth adding in your own words if time allows: one sentence in Discussion naming the light source you used and anything that went wrong.

8 Extra: math sound waves on the USB speaker

optional, 15 min

Plug in the speaker and find its card number, then render and play straight from your recorded run (no Sonic Pi):

```
aplay -l # e.g. "card 1: USB Audio" -> plughw:1,0
python3 waves.py pluck --device plughw:1,0
python3 waves.py fm --device plughw:1,0
```

Notes come from the same random bits as Step 6; what changes is how each note is synthesized.

Pluck (Karplus–Strong string). A delay line of $N = f_s/f_0$ samples is filled with your sensor’s own standardized noise $z_i = (\Delta x_i - \overline{\Delta x})/\sigma$, then recirculated through a two-tap averaging filter:

$$y[n] = \rho \frac{1}{2} (y[n - N] + y[n - N - 1]), \quad \rho = 0.996.$$

The averager is a low-pass filter, so high harmonics die first and the noise burst turns into a decaying plucked string. Every note is excited by a different slice of real measured noise.

FM bells. Frequency modulation with a golden-ratio modulator:

$$y(t) = e(t) \sin(2\pi f_0 t + I(t) \sin(2\pi \varphi f_0 t)), \quad \varphi = \frac{1+\sqrt{5}}{2}.$$

The spectrum has sidebands at $f_0 \pm k\varphi f_0$ with amplitudes $J_k(I)$ (Bessel functions); because φ is irrational they never line up with harmonics, which sounds like a bell. The index $I(t)$ is $4\times$ the rolling mean of $|z|$, so noisier stretches of your data ring brighter.

Both write `waves_pluck.wav` / `waves_fm.wav`; drop one into your demo video.

Troubleshooting

Symptom	Fix
i2cdetect grid empty	I ² C not enabled (Step 1), SDA/SCL swapped, loose socket; power-cycle the Pi
No module named smbus2	<code>sudo apt install -y python3-smbus</code>
ID 0x..., expected 0x50	another chip answers at 0x29; unplug other I ² C devices
TimeoutError	loose wire; reseal the cable, rerun
ch0 stuck near 36 863 / SATURATED	too much light at MAX gain: more paper, flashlight farther
Dark reading moves with room lights	light leak: tape seams, cable notch, hole edges
Randomness tests fail	noise too small (move phone closer) or phone moved; record again
No notes in Sonic Pi	buffer not running, wrong host, or audio going to the wrong output
aplay: silent or wrong speaker	pass <code>--device plughw:N,0</code> with the card number from <code>aplay -l</code>
Report shows SIMULATED banner	old simulated data: <code>rm -r data</code> , re-collect, re-analyze

Files

<code>sensor.py</code>	TSL2591 driver (one-shot reads so no sample is repeated) + simulator
<code>collect.py</code>	<code>check</code> , <code>run</code>
<code>analyze.py</code>	random bits, dice, tests, figures, fills the report
<code>sonify.py</code> , <code>sonic.pi.rb</code>	live OSC music / offline WAV
<code>waves.py</code>	Karplus–Strong and FM renders played on the USB speaker
<code>report/report.tex</code>	the project report (auto-filled)